Installing the NVIDIA GPU Operator on OpenShift

Installing the NVIDIA GPU Operator by using the web console

1. In the OpenShift Container Platform web console, from the side menu, navigate to **Ecosystem** > **System Catalog** and select **All Projects** from the drop down menu. In versions before 4.20, navigate to **Operators** > **OperatorHub** and select **All Projects**.
2. Search for the **NVIDIA GPU Operator**. For additional information, refer to the Red Hat OpenShift Container Platform documentation.
3. Select the **NVIDIA GPU Operator**, click **Install**. In the following screen, click **Install**.

> **Note**
>
> Here, you can select the namespace where you want to deploy the GPU Operator. The suggested namespace to use is the `nvidia-gpu-operator`. You can choose any existing namespace or create a new namespace under **Select a Namespace**.
>
> If you install in any other namespace other than `nvidia-gpu-operator`, the GPU Operator does **not** automatically enable namespace monitoring, and metrics and alerts are **not** collected by Prometheus. If only trusted operators are installed in this namespace, you can manually enable namespace monitoring with this command:
>
> ```
> $ oc label ns/$NAMESPACE_NAME openshift.io/cluster-monitoring=true
> ```

Proceed to Create the cluster policy for the NVIDIA GPU Operator.

Installing the NVIDIA GPU Operator using the CLI

As a cluster administrator, you can install the **NVIDIA GPU Operator** using the OpenShift CLI (`oc`).

1. Create a namespace for the **NVIDIA GPU Operator**.
   a. Create the following `Namespace` custom resource (CR) that defines the `nvidia-gpu-operator` namespace and then save the YAML in the `nvidia-gpu-operator.yaml` file:

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: nvidia-gpu-operator
```

> **Note**
>
> The suggested namespace to use is the `nvidia-gpu-operator`. You can choose any existing namespace or create a new namespace. If you install in any other namespace other than `nvidia-gpu-operator`, the GPU Operator will **not** automatically enable namespace monitoring, and metrics and alerts will **not** be collected by Prometheus.
>
> If only trusted operators are installed in this namespace, you can manually enable namespace monitoring with this command:
>
> ```
> $ oc label ns/$NAMESPACE_NAME openshift.io/cluster-monitoring=true
> ```

   b. Create the namespace by running the following command:

```
$ oc create -f nvidia-gpu-operator.yaml
```

```
namespace/nvidia-gpu-operator created
```

2. Install the **NVIDIA GPU Operator** in the namespace you created in the previous step by creating the following objects:
   a. Create the following `OperatorGroup` CR and save the YAML in the `nvidia-gpu-operatorgroup.yaml` file:

```yaml
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: nvidia-gpu-operator-group
  namespace: nvidia-gpu-operator
spec:
  targetNamespaces:
  - nvidia-gpu-operator
```

b. Create the `OperatorGroup` CR by running the following command:

```
$ oc create -f nvidia-gpu-operatorgroup.yaml
```

```
operatorgroup.operators.coreos.com/nvidia-gpu-operator-group created
```

3. Get the value of `channel`, and store it in a variable:

```
$ CHANNEL=$(oc get packagemanifest gpu-operator-certified -n openshift-marketpl
```

4. Get the `startingCSV` value and store it in a variable:

```
$ STARTING_CSV=$(oc get packagemanifests/gpu-operator-certified -n openshift-ma
```

5. Create the `Subscription` CR using the variables and save the YAML in the `nvidia-gpu-sub.yaml` file:

```
$ cat <<EOF > nvidia-gpu-sub.yaml
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: gpu-operator-certified
  namespace: nvidia-gpu-operator
spec:
  channel: $CHANNEL
  installPlanApproval: Manual
  name: gpu-operator-certified
  source: certified-operators
  sourceNamespace: openshift-marketplace
  startingCSV: $STARTING_CSV
EOF
```

6. Create the subscription object by running the following command:

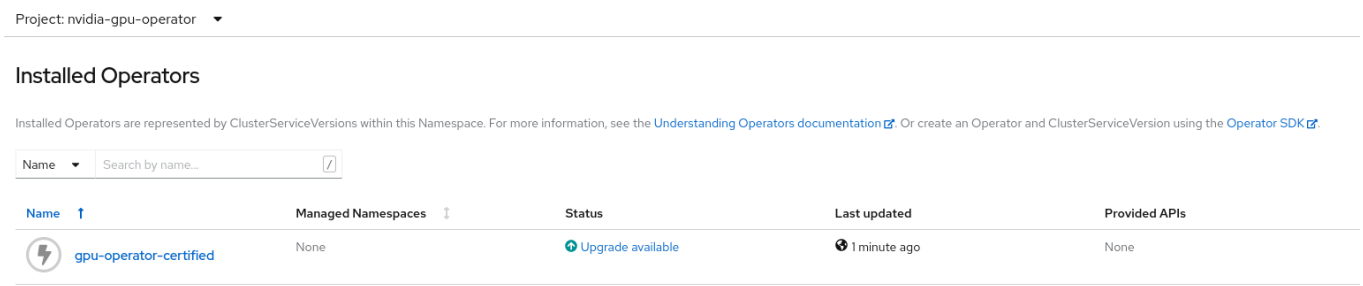
```
$ oc create -f nvidia-gpu-sub.yaml
```

```
subscription.operators.coreos.com/gpu-operator-certified created
```

7. **Optional: Log in to the web console and navigate to the Ecosystem > Installed Operators and select NVIDIA GPU Operator.**

   In versions before 4.20, navigate to **Operators** > **Installed Operators** and select **NVIDIA GPU Operator**.

   In the `Project: nvidia-gpu-operator` the following is displayed:



8. Verify an install plan has been created:

```
$ oc get installplan -n nvidia-gpu-operator
```

Example Output

```
NAME            CSV                                   APPROVAL    APPROVED
install-wwhfj   gpu-operator-certified.v22.9.0        Manual      false
```

9. Approve the install plan using the CLI commands:

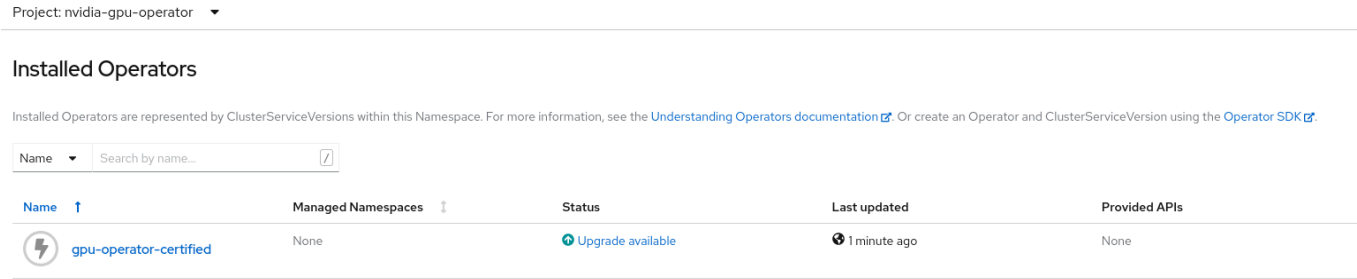
```
$ INSTALL_PLAN=$(oc get installplan -n nvidia-gpu-operator -oname)
```

```
$ oc patch $INSTALL_PLAN -n nvidia-gpu-operator --type merge --patch '{"spec":{
```

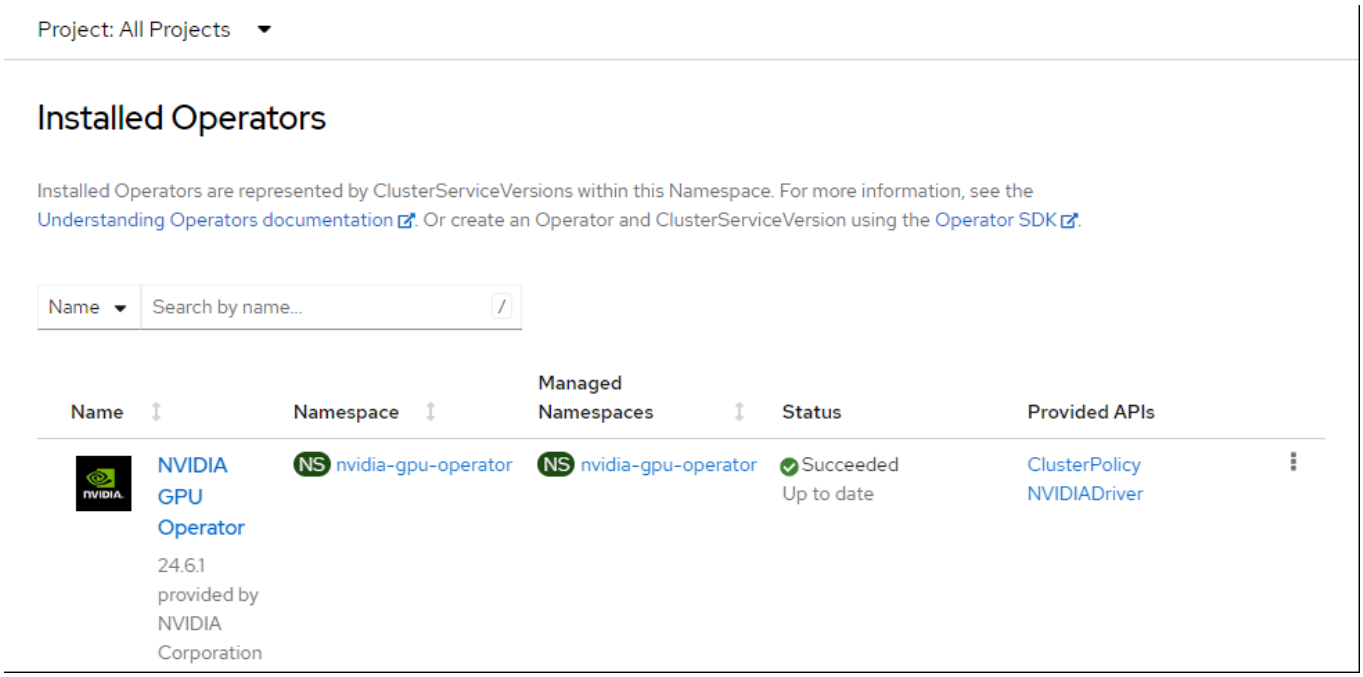
Example Output

```
installplan.operators.coreos.com/install-wwhfj patched
```

10. Alternatively click `Upgrade available` and approve the plan using the web console:



11. Optional: Verify the successful install in the web console. The display changes to:



Create the ClusterPolicy instance

When you install the **NVIDIA GPU Operator** in the OpenShift Container Platform, a custom resource definition for a ClusterPolicy is created. The ClusterPolicy configures the GPU stack, configuring the image names and repository, pod restrictions and credentials, and more.

> **Note**
>
> If you create a ClusterPolicy that contains an empty specification such as `spec{}`, the ClusterPolicy fails to deploy.

As a cluster administrator, you can create a ClusterPolicy using the OpenShift Container Platform CLI or the web console. Also, these steps differ when using **NVIDIA vGPU**. Refer to the appropriate sections below.

Create the cluster policy using the web console

1. In the OpenShift Container Platform web console, from the side menu, select **Ecosystem** > **Installed Operators** (for versions before 4.20, look for **Operators** > **Installed Operators**), and click **NVIDIA GPU Operator**.
2. Select the **ClusterPolicy** tab, then click **Create ClusterPolicy**. The platform assigns the default name *gpu-cluster-policy*.

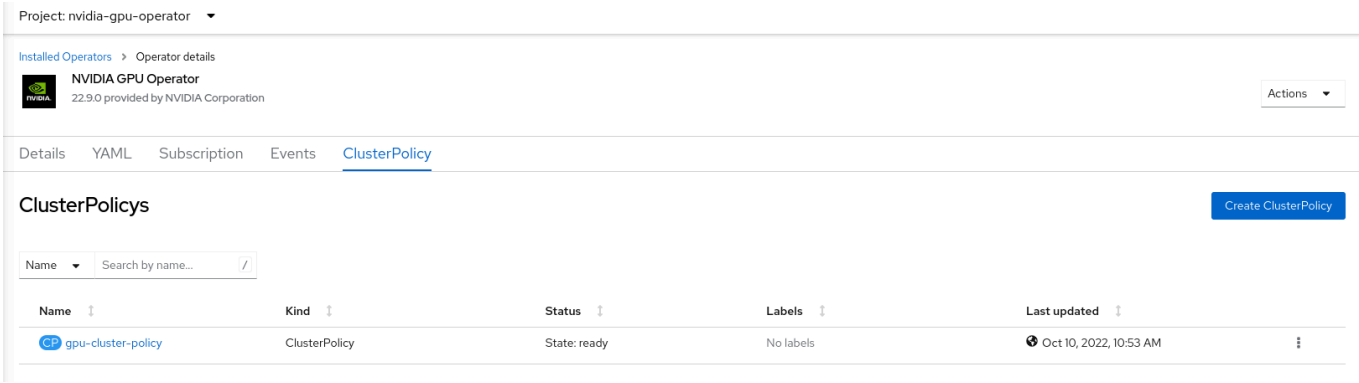
> **Note**
>
> You can use this screen to customize the ClusterPolicy. However, the default values are sufficient to get the GPU configured and running in most cases.

> **Note**
>
> For OpenShift 4.12 with GPU Operator 25.3.1 or later, you must expand the **Driver** section and set the following fields:
>
> - **version**: 570.172.08 (or another supported version)
> - **image**: driver (or another supported image)
> - **repository**: nvcr.io/nvidia (or another supported repository)

3. Click **Create**.

   At this point, the GPU Operator proceeds and installs all the required components to set up the NVIDIA GPUs in the OpenShift 4 cluster. Wait at least 10 to 20 minutes before troubleshooting because this process can take some time to finish.

4. The status of the newly deployed ClusterPolicy *gpu-cluster-policy* for the NVIDIA GPU Operator changes to `State:ready` when the installation succeeds.



Create the cluster policy using the CLI

1. Create the ClusterPolicy:

```
$ oc get csv -n nvidia-gpu-operator $STARTING_CSV -o jsonpath='{.metadata.annot
```

> **Note**
>
> $STARTING_CSV is the value of the `startingCSV` field in the `Subscription` CR sample created in the Installing the NVIDIA GPU Operator using the CLI section.

> **Note**
>
> For OpenShift 4.12 with GPU Operator 25.3.1 or later, modify the `clusterpolicy.json` file to specify `driver.licensingConfig`, `driver.repository`, `driver.image`, `driver.version`, and `driver.imagePullSecrets` (optional). The following snippet is shown as an example. Change values accordingly. Refer to Release Notes for recommended driver versions.

```
"driver": {
     "repository": "nvcr.io/nvidia",
     "image": "driver",
     "version": "570.172.08"
}
```

```
$ oc apply -f clusterpolicy.json
```

```
clusterpolicy.nvidia.com/gpu-cluster-policy created
```

Create the ClusterPolicy instance with NVIDIA vGPU

Prerequisites

Before creating the ClusterPolicy for NVIDIA vGPU, ensure the following:

- Refer to the Using NVIDIA vGPU section for detailed prerequisite steps, including building the vGPU driver container image and obtaining licensing files.
- Create a licensing secret in the `nvidia-gpu-operator` namespace:

```
$ oc create secret generic licensing-config \
    -n nvidia-gpu-operator --from-file=gridd.conf --from-file=client_configurat
```

> **Note**
>
> Using a Kubernetes Secret to store licensing information is the recommended approach. The `configMap` option is deprecated and will be removed in a future release.

- If your private registry requires authentication, create an image pull secret:

```
$ oc create secret docker-registry registry-secret \
    --docker-server=<private-registry-url> \
    --docker-username=<username> \
    --docker-password=<password> \
    --docker-email=<email-id> \
    -n nvidia-gpu-operator
```

Create the cluster policy using the web console

1. In the OpenShift Container Platform web console, from the side menu, select **Ecosystem** > **Installed Operators** and select **NVIDIA GPU Operator**. In versions before 4.20, navigate to **Operators** > **Installed Operators** and select **NVIDIA GPU Operator**.
2. Select the **ClusterPolicy** tab, then click **Create ClusterPolicy**. The platform assigns the default name *gpu-cluster-policy*.
3. Under the **NVIDIA GPU/vGPU Driver config** section fill in the following information
   - Specify the `secretName` as the name of the licensing `Secret` created during the prerequisite steps for NVIDIA vGPU (for example, `licensing-config`).

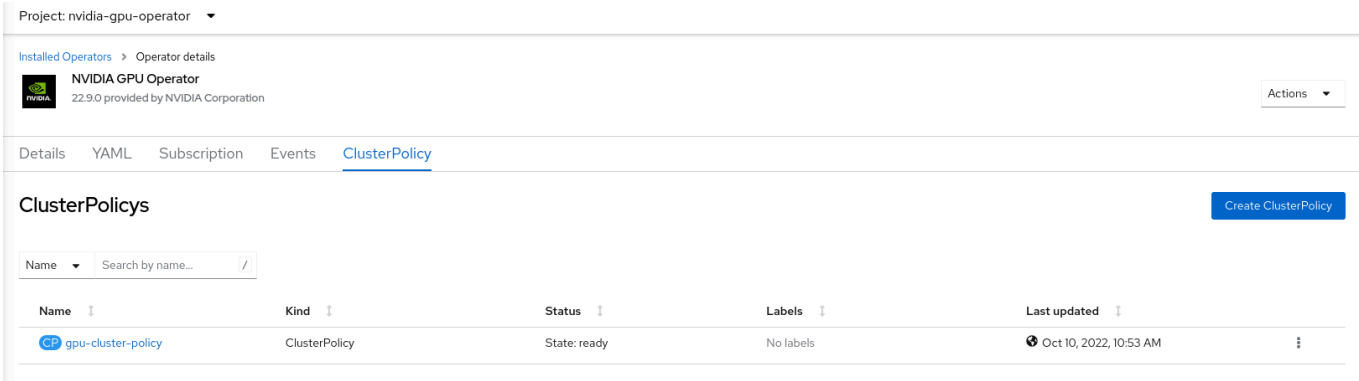
> **Note**
>
> The `ConfigMap` option is deprecated. Use a Kubernetes `Secret` to store licensing information instead.

- Specify the `repository` path, `image` name, and NVIDIA vGPU driver `version`. If the registry is not public, specify the `imagePullSecret` created during the prerequisite step under the **Driver** advanced configurations section.

4. Click **Create**.

   At this point, the GPU Operator proceeds and installs all the required components to set up the NVIDIA GPUs in the OpenShift 4 cluster. Wait at least 10 to 20 minutes before troubleshooting because this process can take some time to finish.

5. The status of the newly deployed ClusterPolicy *gpu-cluster-policy* for the NVIDIA GPU Operator changes to `State:ready` when the installation succeeds.



Create the cluster policy using the CLI

1. Create the ClusterPolicy:

```
$ oc get csv -n nvidia-gpu-operator $STARTING_CSV -o jsonpath='{.metadata.annot
```

> **Note**
>
> $STARTING_CSV is the value of the `startingCSV` field in the `Subscription` CR sample created in the Installing the NVIDIA GPU Operator using the CLI section.

2. Modify the `clusterpolicy.json` file to specify `driver.licensingConfig`, `driver.repository`, `driver.image`, `driver.version`, and `driver.imagePullSecrets` created during the prerequisite steps. The following snippet is shown as an example. Change values accordingly.

```
"driver": {
     "repository": "<repository-path>",
     "image": "driver",
     "imagePullSecrets": ["registry-secret"],
     "licensingConfig": {
       "secretName": "licensing-config",
       "nlsEnabled": true
     },
     "version": "580.95.05"
}
```

> **Note**
>
> Using `secretName` to reference a Kubernetes Secret is the recommended approach. The `configMapName` option is deprecated and will be removed in a future release.

3. Apply the ClusterPolicy:

```
$ oc apply -f clusterpolicy.json
```

```
clusterpolicy.nvidia.com/gpu-cluster-policy created
```

Verify the successful installation of the NVIDIA GPU Operator

Verify the successful installation of the NVIDIA GPU Operator as shown here:

1. Run the following command to view these new pods and daemonsets:

```
$ oc get pods,daemonset -n nvidia-gpu-operator
```

```
NAME                                                      READY   STATUS      R
pod/gpu-feature-discovery-c2rfm                           1/1     Running     0
pod/gpu-operator-84b7f5bcb9-vqds7                         1/1     Running     0
pod/nvidia-container-toolkit-daemonset-pgcrf              1/1     Running     0
pod/nvidia-cuda-validator-p8gv2                           0/1     Completed   0
pod/nvidia-dcgm-exporter-kv6k8                            1/1     Running     0
pod/nvidia-dcgm-tpsps                                     1/1     Running     0
pod/nvidia-device-plugin-daemonset-gbn55                  1/1     Running     0
pod/nvidia-device-plugin-validator-z7ltr                  0/1     Completed   0
pod/nvidia-driver-daemonset-410.84.202203290245-0-xxgdv   2/2     Running     0
pod/nvidia-node-status-exporter-snmsm                     1/1     Running     0
pod/nvidia-operator-validator-6pfk6                       1/1     Running     0

NAME                                                         DESIRED   CURREN
daemonset.apps/gpu-feature-discovery                         1         1
daemonset.apps/nvidia-container-toolkit-daemonset            1         1
daemonset.apps/nvidia-dcgm                                   1         1
daemonset.apps/nvidia-dcgm-exporter                          1         1
daemonset.apps/nvidia-device-plugin-daemonset                1         1
daemonset.apps/nvidia-driver-daemonset-410.84.202203290245-0 1         1
daemonset.apps/nvidia-mig-manager                            0         0
daemonset.apps/nvidia-node-status-exporter                   1         1
daemonset.apps/nvidia-operator-validator                     1         1
```

The `nvidia-driver-daemonset` pod runs on each worker node that contains a supported NVIDIA GPU.

> **Note**
>
> When the Driver Toolkit is active, the `DaemonSet` is named `nvidia-driver-daemonset-<RHCOS-version>`. Where `RHCOS-version` equals `<OCP XY>.<RHEL XY>.<related date YYYYMMDDHHSS-0`. The pods of the `DaemonSet` are named `nvidia-driver-daemonset-<RHCOS-version>-<UUID>`.

Cluster monitoring

The GPU Operator generates GPU performance metrics (DCGM-export), status metrics (node-status-exporter) and node-status alerts. For OpenShift Prometheus to collect these metrics, the namespace hosting the GPU Operator must have the label `openshift.io/cluster-monitoring=true`.

When the GPU Operator is installed in the suggested `nvidia-gpu-operator` namespace, the GPU Operator automatically enables monitoring if the `openshift.io/cluster-monitoring`

label is not defined. If the label is defined, the GPU Operator will not change its value.

Disable cluster monitoring in the `nvidia-gpu-operator` namespace by setting `openshift.io/cluster-monitoring=false`:

```
$ oc label ns/nvidia-gpu-operator openshift.io/cluster-monitoring=false
```

If the GPU Operator is not installed in the suggested namespace, the GPU Operator will not automatically enable monitoring. Set the label manually as shown:

```
$ oc label ns/$NAMESPACE openshift.io/cluster-monitoring=true
```

> **Note**
>
> Only do this if trusted operators are installed in this namespace.

Logging

The `nvidia-driver-daemonset` pod has two containers.

1. Run the following to examine the logs associated with the `nvidia-driver-ctr`:

   > **Note**
   >
   > This log shows the main container waiting for the driver binary, and loading it in memory.

   ```
   $ oc logs -f nvidia-driver-daemonset-410.84.202203290245-0-xxgdv -n nvidia-gpu-
   ```

2. Run the following to examine the logs associated with the `openshift-driver-toolkit-ctr`:

   > **Note**
   >
   > This log shows the driver being built.

```
$ oc logs -f nvidia-driver-daemonset-410.84.202203290245-0-xxgdv -n nvidia-gpu-
```

Running a sample GPU Application

Run a simple CUDA VectorAdd sample that adds two vectors together to ensure the GPUs have bootstrapped correctly.

1. Run the following:

```
$ cat << EOF | oc create -f -

apiVersion: v1
kind: Pod
metadata:
  name: cuda-vectoradd
spec:
  restartPolicy: OnFailure
  containers:
  - name: cuda-vectoradd
    image: "nvcr.io/nvidia/k8s/cuda-sample:vectoradd-cuda12.5.0-ubi8"
    resources:
      limits:
        nvidia.com/gpu: 1
EOF
```

```
pod/cuda-vectoradd created
```

2. Check the logs of the container:

```
$ oc logs cuda-vectoradd
```

```
[Vector addition of 50000 elements]
Copy input data from the host memory to the CUDA device
CUDA kernel launch with 196 blocks of 256 threads
Copy output data from the CUDA device to the host memory
Test PASSED
Done
```

Getting information about the GPU

The `nvidia-smi` command shows memory usage, GPU utilization, and the temperature of the GPU. Test the GPU access by running the popular `nvidia-smi` command within the pod.

To view GPU utilization, run `nvidia-smi` from a pod in the GPU Operator daemonset.

1. Change to the nvidia-gpu-operator project:

```
$ oc project nvidia-gpu-operator
```

2. Run the following command to view these new pods:

```
$ oc get pod -owide -lopenshift.driver-toolkit=true
```

```
NAME                                                    READY   STATUS    RESTART
nvidia-driver-daemonset-410.84.202203290245-0-xxgdv     2/2     Running   0
```

> **Note**
>
> With the pod and node name, run the `nvidia-smi` command on the correct node.

3. Run the `nvidia-smi` command within the pod:

```
$ oc exec -it nvidia-driver-daemonset-410.84.202203290245-0-xxgdv -- nvidia-smi
```

```
Defaulted container "nvidia-driver-ctr" out of: nvidia-driver-ctr, openshift-dr
Mon Apr 11 15:02:23 2022
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 510.47.03    Driver Version: 510.47.03    CUDA Version: 11.6     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
```

Privacy Policy | Your Privacy Choices | Terms of Service | Accessibility | Corporate Policies | Product Security | Contact

Copyright © 2020-2026, NVIDIA Corporation.